

CO3-AT1-Assignment



Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

QUESTIONS

Q1 Banking Transaction System – Exception Handling
10 marks

Q2 Hospital Patient Registration – User-Defined Exception
10 marks

Q3 E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 Online Food Delivery – Multiple Threads and Execution Order
10 marks

Q5 Manufacturing Plant – Producer-Consumer Problem
10 marks

5/5 answered

Q1 Banking Transaction System – Exception Handling

10 marks

Develop a Java program for an ATM withdrawal system. The program accepts the account balance and withdrawal amount. The program must:

- Accept the current account balance and withdrawal amount.
- Throw a user-defined `InsufficientBalanceException` when the withdrawal amount exceeds the balance.
- Handle withdrawal amounts that are zero or negative.
- Handle invalid numeric input using a suitable built-in exception.
- Display a meaningful transaction result.
- Use try-catch-finally so that the transaction completion message is always displayed.

Sample Test Cases:

Input / Scenario	Expected Result	Purpose
10000 3000	Normal case	
10000 12000	Boundary / exception case	
10000 -500	Invalid input	
abc 3000	Invalid input	

Your Code

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
```

Input
10000
3000



Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

QUESTIONS

Q1 Manufacturing Plant – Producer-Consumer Problem
10 marks

5/5 answered

Your Code

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         try{
6             double balance=sc.nextDouble();
7             double amount=sc.nextDouble();
8             if(amount<=0)
9                 throw new IllegalArgumentException("Invalid withdraw");
10            if(amount>balance)
11                throw new InsufficientBalanceException("Insufficient");
12            System.out.println("Withdrawal successful");
13            System.out.println("Remaining balance: "+(balance-amount));
14        }catch(InputMismatchException e){
15            System.out.println("Invalid numeric input");
16        }catch(InsufficientBalanceException e){
17            System.out.println(e.getMessage());
18        }catch(IllegalArgumentException e){
19            System.out.println(e.getMessage());
20        }finally{
21            System.out.println("Transaction completed");
22        }
23    }
24 }
25 class InsufficientBalanceException extends Exception{
26     InsufficientBalanceException(String msg){
27         super(msg);
```

Input
10000
3000

Output
Withdrawal successful
Remaining balance: 7000.0
Transaction completed


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING

VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

QUESTIONS

Q1
 Banking Transaction System – Exception Handling
 10 marks

Q2
 Hospital Patient Registration – User-Defined Exception
 10 marks

Q3
 E-Commerce Order Processing – Built-in Exceptions
 10 marks

Q4
 Online Food Delivery – Multiple Threads and Execution Order
 10 marks

Q5
 Manufacturing Plant – Producer-Consumer Problem
 10 marks

5/5 answered

Q2 Hospital Patient Registration – User-Defined Exception
10 marks

Develop a Java program for a hospital patient registration system. The program accepts the patient's name, age, and body temperature.

Create suitable user-defined exception classes for invalid age and invalid temperature. Validate the input before registration. Use try-catch-finally and display a registration completion message from finally.

Sample Test Cases:

Input / Scenario Expected Result Purpose

Arun, 35, 98.6 Patient registered successfully Normal case

Meena, 12, 98.4 InvalidAgeException Boundary / exception case

Ravi, 45, 105.2 InvalidTemperatureException Invalid input

Kumar, -2, 98.1 InvalidAgeException Invalid input

Your Code

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         try{
6             String name=sc.next();
7             int age=sc.nextInt();
8             double temp=sc.nextDouble();
9             if(age<18||age>100)
10                throw new InvalidAgeException("InvalidAgeException")
11

```

Input

Arun 35 98.6

Output

Patient registered successfully
Registration completed


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING

VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

5/5 answered

```

7         int age=sc.nextInt();
8         double temp=sc.nextDouble();
9         if(age<18||age>100)
10            throw new InvalidAgeException("InvalidAgeException")
11         if(temp<95||temp>100)
12            throw new InvalidTemperatureException("InvalidTemperatureException")
13         System.out.println("Patient registered successfully");
14
15     }catch(InvalidAgeException e){
16         System.out.println(e.getMessage());
17     }catch(InvalidTemperatureException e){
18         System.out.println(e.getMessage());
19     }catch(InputMismatchException e){
20         System.out.println("Invalid input");
21     }finally{
22         System.out.println("Registration completed");
23     }
24 }
25 }
26 }
27 class InvalidAgeException extends Exception{
28     InvalidAgeException(String msg){
29         super(msg);
30     }
31 }
32 class InvalidTemperatureException extends Exception{
33     InvalidTemperatureException(String msg){
34         super(msg);
35     }
36 }

```

Patient registered successfully
Registration completed


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

QUESTIONS

Q1 
 Banking Transaction System – Exception Handling
 10 marks

Q2 
 Hospital Patient Registration – User-Defined Exception
 10 marks

Q3 
 E-Commerce Order Processing – Built-in Exceptions
 10 marks

Q4 
 Online Food Delivery – Multiple Threads and Execution Order
 10 marks

Q5 
 Manufacturing Plant – Producer-Consumer Problem
 10 marks

5/5 answered

Q3 E-Commerce Order Processing – Built-in Exceptions 10 marks

Develop a Java program for an online shopping application. Store product names and prices in arrays and accept a product index and quantity from the customer.
 The program must:

- Calculate total cost as quantity × price.
- Handle invalid product indices.
- Handle invalid numeric input.
- Reject zero or negative quantities.
- Use appropriate built-in exception classes.
- Continue execution after handling an invalid transaction.

 Sample Test Cases:
 Input / Scenario Expected Result Purpose
 1, 2 Valid product; total cost calculated Normal case
 5, 2 ArrayIndexOutOfBoundsException handled Boundary / exception case
 2, -1 Invalid quantity Invalid input
 abc, 2 InputMismatchException handled Invalid input

Your Code


```

1 import java.util.*;
2 public class Main{
3     public static void main(String[]args) {
4         Scanner sc=new Scanner(System.in);
5         String[] product={"Laptop","Mobile","Headphones"};
        
```

Input

1 2


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

10 marks

5/5 answered

Your Code

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[]args){
4         Scanner sc=new Scanner(System.in);
5         String[] product={"Laptop","Mobile","Headphones"};
6         double[] price={50000,20000,2000};
7         try{
8             int index=sc.nextInt();
9             int quantity=sc.nextInt();
10            if(quantity<=0)
11                throw new IllegalArgumentException("Invalid quantity");
12            System.out.println("Product: "+product[index]);
13            System.out.println("Total cost: "+(price[index]*quantity));
14        }catch(ArrayIndexOutOfBoundsException e){
15            System.out.println("ArrayIndexOutOfBoundsException handled");
16        }catch(InputMismatchException e){
17            System.out.println("InputMismatchException handled");
18        }catch(IllegalArgumentException e){
19            System.out.println(e.getMessage());
20        }
21    }
22 }
        
```

Input

1 2

Output

Product: Mobile
 Total cost: 40000.0

 Submitted



Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING

VANGIMALLA BALA SIVA PRASAD REDDY
 192424316RA

QUESTIONS

Q1
Banking Transaction System – Exception Handling
10 marks

Q2
Hospital Patient Registration – User-Defined Exception
10 marks

Q3
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4
Online Food Delivery – Multiple Threads and Execution Order
10 marks

Q5
Manufacturing Plant – Producer-Consumer Problem
10 marks

5/5 answered

Q4 Online Food Delivery – Multiple Threads and Execution Order

10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution

Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design

Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[] args) throws Exception{
4         Scanner sc=new Scanner(System.in);

```

Input

Pizza UPI AgentA



Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING

VANGIMALLA BALA SIVA PRASAD REDDY
 192424316RA

```

5         String order=sc.next();
6         String payment=sc.next();
7         String delivery=sc.next();
8
9         OrderThread t1=new OrderThread();
10        PaymentThread t2=new PaymentThread(payment);
11        DeliveryThread t3=new DeliveryThread(payment);
12        t1.start();
13        t1.join();
14        t2.start();
15        t2.join();
16        t3.start();
17        t3.join();
18    }
19 }
20 class OrderThread extends Thread{
21     public void run(){
22         System.out.println("Order processing completed");
23     }
24 }
25 class PaymentThread extends Thread{
26     String payment;
27     PaymentThread(String payment){
28         this.payment=payment;
29     }
30     public void run(){
31         if(payment.equalsIgnoreCase("Failed"))
32             System.out.println("Payment failed");
33         else

```

Output

```

Order processing completed
Payment processing completed
Delivery assignment completed

```


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

```

23     }
24 }
25 class PaymentThread extends Thread{
26     String payment;
27     PaymentThread(String payment){
28         this.payment=payment;
29     }
30     public void run(){
31         if(payment.equalsIgnoreCase("Failed"))
32             System.out.println("Payment failed");
33         else
34             System.out.println("Payment processing completed");
35     }
36 }
37 class DeliveryThread extends Thread{
38     String payment;
39     DeliveryThread(String payment){
40         this.payment=payment;
41     }
42     public void run(){
43         if(payment.equalsIgnoreCase("Failed"))
44             System.out.println("Delivery assignment cancelled");
45         else
46             System.out.println("Delivery assignment completed");
47     }
48 }
        
```

Submitted


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

QUESTIONS

Q1
 Banking Transaction System –
 Exception Handling
 10 marks

Q2
 Hospital Patient Registration –
 User-Defined Exception
 10 marks

Q3
 E-Commerce Order Processing
 – Built-in Exceptions
 10 marks

Q4
 Online Food Delivery – Multiple
 Threads and Execution Order
 10 marks

Q5
 Manufacturing Plant –
 Producer-Consumer Problem
 10 marks

5/5 answered

Q5 Manufacturing Plant – Producer-Consumer Problem
10 marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product. Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:

- Make the producer wait when the buffer is full.
- Make the consumer wait when the buffer is empty.
- Prevent duplicate consumption.
- Ensure that products are consumed only after production.
- Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution

Consumer starts first Consumer waits until a product is available Concurrency / design

Buffer capacity = 1; Products = P1, P2 Producer waits while P1 remains unconsumed Exception / boundary

Your Code

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[]args) throws Exception{
4         Buffer b=new Buffer();
        
```

Input

stdin input

```

4      Buffer b=new Buffer();
5      Thread producer=new Thread(){
6          public void run(){
7              try{
8                  b.produce("P1");
9                  b.produce("P2");
10                 b.produce("P3");
11             }catch(Exception e){
12             }
13         }
14     };
15     Thread consumer=new Thread(){
16         public void run(){
17             try{
18                 b.getProduct();
19                 b.getProduct();
20                 b.getProduct();
21             }catch(Exception e){
22             }
23         }
24     };
25     producer.start();
26     consumer.start();
27
28     producer.join();
29     consumer.join();
30 }
31 }
32 class Buffer{

```

Output

```

Produced: P1
Consumed: P1
Produced: P2
Consumed: P2
Produced: P3
Consumed: P3

```

```

24     };
25     producer.start();
26     consumer.start();
27
28     producer.join();
29     consumer.join();
30 }
31 }
32 class Buffer{
33     String product;
34     boolean full=false;
35     synchronized void produce(String p) throws Exception{
36         while(full)
37             wait();
38         product=p;
39         full=true;
40         System.out.println("Produced: "+product);
41         notify();
42     }
43     synchronized void getProduct() throws Exception{
44         while(!full)
45             wait();
46         System.out.println("Consumed: "+product);
47         full=false;
48         notify();
49     }
50 }

```

Submitted